

On Weak Bisimilarities in CCSK

Baptiste Vallée¹[0009-0002-1268-3761] and Ivan Lanese²[0000-0003-2527-9995]
 baptistevalleedupont@gmail.com, ivan.lanese@gmail.com

¹ École Normale Supérieure Paris-Saclay (France)

² Olas Team, University of Bologna/INRIA (Italy)

This is a pre-copy-editing, author-produced PDF of an article accepted for publication in RC 2026 following peer review. The definitive publisher-authenticated version is available online at <https://link.springer.com/chapter/10.1007/978-3-032-30839-9>

Abstract. In the context of CCSK, a reversible extension of CCS, we study different notions of bisimilarity (strong/weak, forward-only/reversible) and highlight their differences and commonalities. In particular, for the weak reversible case, not previously studied in the literature, we propose two variants, dubbed directional and mixed bisimilarity, depending on whether τ actions should be in the same direction (forward/backward) as the action being matched or not. We show, in particular, that mixed bisimilarity is a congruence and completely abstracts away from τ actions.

Keywords: CCS, CCSK, Reversible Computation, Weak Bisimilarity, Behavioral Theory

1 Introduction

Building concurrent systems is challenging due to the complexity of reasoning about numerous possible interleavings, yet concurrency is essential in modern systems like the Internet, cloud computing, and parallel processing. Reversible computing, which allows systems to execute both forwards and backwards, recovering past states, has significant applications in low-energy computing [9], simulation [5], biological modeling [4,19], and program debugging [8,13,10]. Many of these applications involve concurrent systems, leading to the development of reversible extensions of concurrent process calculi such as CCS [7,18] and the π -calculus [6], and even of concurrent programming languages such as Erlang [11] and Go [17].

A main notion in the theory of process calculi is the notion of bisimilarity [20], allowing one to prove two processes equivalent, e.g., to prove an implementation equivalent to a more abstract specification. In particular, bisimilarity requires equivalent processes to be able to match each other actions, and in doing so going to processes which are still equivalent. While strong and weak bisimilarity (weak bisimilarity differs from the strong one as the former abstracts away from internal actions, focusing only on interactions with the context) have been extensively studied in concurrent systems, the literature lacks an analysis of weak bisimilarities in a reversible setting. Our study addresses this gap

by investigating the relationships between different notions of bisimilarity (strong/weak, forward-only/reversible) in the context of CCSK [18], a causal-consistent reversible extension of Milner CCS [15]. In particular, in the definition of weak reversible bisimilarity, a main decision is whether auxiliary τ actions (representing internal steps) allowed in the simulation of some action α need to be in the same direction as α or not. The two alternatives lead to different equivalences.

We consider this work as a first step in the exploration of weak bisimilarity in a reversible setting, paving the way for a deeper exploration in the future. We claim as our main contributions the proposal of two notions of weak reversible bisimilarity (mixed bisimilarity in Definition 3.13 and directional bisimilarity in Definition 3.15, both in Section 3), the study of the relations between different notions of bisimilarity in CCSK (Section 4), and the study of which of these notions are congruences (Section 5). In particular, we show that mixed bisimilarity is a congruence (Theorem 5.6) and completely abstracts away from τ actions (Proposition 5.5 and Theorem 5.7). Another surprising result is that extending CCS bisimilarities to CCSK gives equivalences which are not congruences (Proposition 5.2), even when they are congruences in CCS, as in the case of strong bisimilarity.

2 CCSK

In this section we recall the main elements of CCSK, while referring to [18] for further details. We assume an infinite set of *Names* $\mathcal{A}$, ranged over by $a, b, c, \dots$, and a disjoint infinite set of *Co-names* $\overline{\mathcal{A}}$, ranged over by $\bar{a}, \bar{b}, \bar{c}, \dots$, where $\bar{\cdot}$ is an operator such that $\bar{\bar{a}} = a$. We call *actions* the elements of $\mathcal{A} \cup \overline{\mathcal{A}} \cup \{\tau\}$ where $\tau \notin \mathcal{A}$ and $\bar{\tau}$ is undefined, ranged over by $\alpha, \beta, \dots$. Intuitively, names represent input actions, co-names represent output actions, and τ is an internal synchronisation. CCS processes, which we shall also call *standard* processes, are given by:

$$P, Q := 0 \mid \alpha.P \mid P + Q \mid P \mid Q \mid (\nu a)P$$

Intuitively, 0 is the inactive process, $\alpha.P$ is a process that performs action α and continues as P , $P + Q$ is nondeterministic choice, $P \mid Q$ is parallel composition and restriction $(\nu a)P$ binds name a and the corresponding co-name $\bar{a}$ inside P . A name is *bound* if it is inside the scope of a restriction operator, *free* otherwise. Function $\text{fn}(P)$ computes the set of free names in process P . We set this convention: unary operators bind stronger than binary operators.

CCSK extends CCS with the possibility of executing backwards. In order to remember which input interacted with which output while going forwards, fresh *keys* are created at each forward step, and the same key is used to label an input and the corresponding output during a synchronisation.

We denote the set of keys by **Keys**, ranged over by $m, n, k, \dots$. Prefixes, ranged over by π , are of the form $\alpha[m]$ or α . The former denotes that α has already been executed, the latter that it has not.

CCSK processes are given by:

$$P, Q := 0 \parallel \pi.P \parallel P + Q \parallel P \mid Q \parallel (\nu a)P$$

hence they are like CCS processes but for the fact that prefixes may be labelled with a key. In the following, we may drop trailing 0s.

Definition 2.1 (Context). *A CCSK context is a process with a hole, as generated by the grammar below:*

$$C := \bullet \parallel \pi.C \parallel C + Q \parallel P + C \parallel C \mid Q \parallel P \mid C \parallel (\nu a)C$$

We denote with $C[P]$ the process obtained by replacing $\bullet$ with P inside C .

We use predicate $\text{std}(P)$ to mean that P is standard, that is none of its actions has been executed, hence it has no keys. We assume function $\text{toStd}(P)$ that takes a CCSK process P and gives back the standard process obtained by removing all keys from P .

We take from [12, Def. 2.1] the notions of free and bound keys.

Definition 2.2 (Free and bound keys). *A key k is bound in a process X iff it occurs either twice, attached to complementary prefixes, or once, attached to a τ prefix. A key k is free if it occurs once, attached to a non- τ prefix.*

Figure 1 shows the forward rules of CCSK. Backward rules in Figure 2 are obtained from forward rules by reversing the direction of transitions. Both relations rely on a definition of structural congruence allowing one to α -convert bound keys, applicable only at top level (this condition is needed to ensure that there are no other occurrences of n in the context):

$$P \equiv P[n/m] \quad m \text{ bound in } P, n \notin \text{keys}(P)$$

Rule (TOP) allows a prefix to execute. The rule generates a key m . Freshness of m is guaranteed by the side conditions of the other rules (cf. rule (PAR)). Rule (PREFIX) states that an executed prefix does not block execution. The two rules for (CHOICE) and the two for (PAR) allow processes to execute inside a choice or a parallel composition. The side condition of rule (CHOICE) ensures that at most one branch can execute. Rule (SYNCH) allows two complementary actions to synchronise producing a τ . The key of the two actions needs to be the same. Rule (RES) allows an action which does not involve the restricted name to propagate through restriction.

The forward semantics of a CCSK process is the smallest relation $\rightarrow_f$ closed under the rules in Figure 1. Analogously, its backward semantics is the smallest relation $\rightarrow_r$ closed under the rules in Figure 2. The semantics is the union of the two relations. From

$$\begin{array}{l}
\text{(TOP)} \quad \frac{\text{std}(P)}{\alpha.P \xrightarrow{\alpha[m]}_f \alpha[m].P} \quad \text{(PREFIX)} \quad \frac{P \xrightarrow{\beta[n]}_f P'}{\alpha[m].P \xrightarrow{\beta[n]}_f \alpha[m].P'} \quad m \neq n \\
\text{(CHOICE)} \quad \frac{P \xrightarrow{\alpha[m]}_f P' \quad \text{std}(Q)}{P + Q \xrightarrow{\alpha[m]}_f P' + Q} \quad \frac{Q \xrightarrow{\alpha[m]}_f Q' \quad \text{std}(P)}{P + Q \xrightarrow{\alpha[m]}_f P + Q'} \\
\text{(PAR)} \quad \frac{P \xrightarrow{\alpha[m]}_f P' \quad m \notin \text{keys}(Q)}{P \mid Q \xrightarrow{\alpha[m]}_f P' \mid Q} \quad \frac{Q \xrightarrow{\alpha[m]}_f Q' \quad m \notin \text{keys}(P)}{P \mid Q \xrightarrow{\alpha[m]}_f P \mid Q'} \\
\text{(SYNCH)} \quad \frac{P \xrightarrow{\alpha[m]}_f P' \quad Q \xrightarrow{\bar{\alpha}[m]}_f Q'}{P \mid Q \xrightarrow{\tau[m]}_f P' \mid Q'} \quad (\alpha \neq \tau) \\
\text{(RES)} \quad \frac{P \xrightarrow{\alpha[m]}_f P'}{(\nu a)P \xrightarrow{\alpha[m]}_f (\nu a)P'} \quad \alpha \notin \{a, \bar{a}\} \\
\text{(EQUIV)} \quad \frac{P \equiv Q \quad Q \xrightarrow{\alpha[m]}_f Q' \quad Q' \equiv P'}{P \xrightarrow{\alpha[m]}_f P'} \quad \begin{array}{l} \text{applicable only} \\ \text{at top level} \end{array}
\end{array}$$

Fig. 1. Forward SOS rules for CCSK

now on, we let ϑ range over $\alpha[m]$ and μ range over $\alpha[m]$ with $\alpha \neq \tau$. Let $x, y, \dots$ range over the set of directions $\{f, r\}$, for forward and reverse.

As standard in reversible computing (see, e.g., [18] or the notion of coherent process in [7]), all the developments consider only processes reachable from a standard process.

Definition 2.3 (Reachable process). *A process Q is reachable iff there exists a standard process P and a finite sequence of transitions from P to Q .*

3 Bisimilarities

In this section we introduce the various notions of bisimilarity we study. We start from CCS bisimilarities, and then move to bisimilarities specific for CCSK. Note that CCS can be seen as a subset of CCSK, considering only standard processes and only forward semantics. Hence, we will extend CCS bisimulations to CCSK by just considering forward transitions of CCSK terms. As a consequence, history becomes inaccessible, hence ideally irrelevant. However, requiring that a transition is matched by a transition with identical label would leak information on which keys are used in the history, as shown by the following example.

Example 3.1. We have $b \xrightarrow{b[n]}_f b[n]$ while no transition with the same label is enabled from $a[n].b$. The second process hence cannot match the transition of the first one in the bisimulation game if equality of labels is required. Hence, keys of forward transitions leak information about which keys are used in the history.

In order to avoid this issue, we will remove keys from the labels. Hence, we define CCS semantics for CCSK processes as follows:

$$\begin{array}{l}
\text{(BK-TOP)} \quad \frac{\text{std}(P)}{\alpha[m].P \xrightarrow{\alpha[m]}_r \alpha.P} \quad \text{(BK-PREFIX)} \quad \frac{P \xrightarrow{\beta[n]}_r P'}{\alpha[m].P \xrightarrow{\beta[n]}_r \alpha[m].P'} \quad m \neq n \\
\text{(BK-CHOICE)} \quad \frac{P \xrightarrow{\alpha[m]}_r P' \quad \text{std}(Q)}{P + Q \xrightarrow{\alpha[m]}_r P' + Q} \quad \frac{Q \xrightarrow{\alpha[m]}_r Q' \quad \text{std}(P)}{P + Q \xrightarrow{\alpha[m]}_r P + Q'} \\
\text{(BK-PAR)} \quad \frac{P \xrightarrow{\alpha[m]}_r P' \quad m \notin \text{keys}(Q)}{P \mid Q \xrightarrow{\alpha[m]}_r P' \mid Q} \quad \frac{Q \xrightarrow{\alpha[m]}_r Q' \quad m \notin \text{keys}(P)}{P \mid Q \xrightarrow{\alpha[m]}_r P \mid Q'} \\
\text{(BK-SYNCH)} \quad \frac{P \xrightarrow{\alpha[m]}_r P' \quad Q \xrightarrow{\bar{\alpha}[m]}_r Q'}{P \mid Q \xrightarrow{\tau[m]}_r P' \mid Q'} \quad (\alpha \neq \tau) \\
\text{(BK-RES)} \quad \frac{P \xrightarrow{\alpha[m]}_r P'}{(\nu a)P \xrightarrow{\alpha[m]}_r (\nu a)P'} \quad \alpha \notin \{a, \bar{a}\} \\
\text{(BK-EQUIV)} \quad \frac{P \equiv Q \quad Q \xrightarrow{\alpha[m]}_r Q' \quad Q' \equiv P'}{P \xrightarrow{\alpha[m]}_r P'} \quad \begin{array}{l} \text{applicable only} \\ \text{at top level} \end{array}
\end{array}$$

Fig. 2. Reverse SOS rules for CCSK

Definition 3.2 (CCS semantics for CCSK processes). *Given a CCSK process P , $P \xrightarrow{\alpha} P'$ iff there is k such that $P \xrightarrow{\alpha[k]}_f P'$.*

We show now that the semantics above, defined on all CCSK processes, is indeed strictly related to the classical semantics of CCS defined in [14, Chapter 5]³, which we denote as $\rightarrow_{ccs}$. Let $\text{delHist}()$ be the function that extracts the standard part of a process.

Definition 3.3 ($\text{delHist}()$). *The $\text{delHist}()$ function is inductively defined as follows:*

$$\begin{aligned}
\text{delHist}(P) &= P \text{ if } \text{std}(P) \\
\text{delHist}(a[n].P) &= \text{delHist}(P) \\
\text{delHist}(P + Q) &= \text{delHist}(P) \text{ if } \neg \text{std}(P) \\
\text{delHist}(P + Q) &= \text{delHist}(Q) \text{ if } \neg \text{std}(Q) \\
\text{delHist}(P \mid Q) &= \text{delHist}(P) \mid \text{delHist}(Q) \\
\text{delHist}((\nu a)P) &= (\nu a)\text{delHist}(P)
\end{aligned}$$

The following proposition holds.

Proposition 3.4. *Let P, P' be CCSK processes, and Q a CCS process. If $P \xrightarrow{\alpha} P'$ then $\text{delHist}(P) \xrightarrow{\alpha}_{ccs} \text{delHist}(P')$. If $\text{delHist}(P) \xrightarrow{\alpha}_{ccs} Q$ then there exists P' in CCSK such that $P \xrightarrow{\alpha} P'$ and $Q = \text{delHist}(P')$.*

Proof. By rule inspection. □

³ Actually, the semantics in [14, Chapter 5] includes additional operators, as well as value passing. We consider its restriction to the operators we use in CCSK.

Bisimilarities for CCS We start with the classical notion of CCS bisimilarity, extended as mentioned above.

Definition 3.5 (Strong Bisimulation). *A symmetric relation $\mathcal{R}$ on CCSK processes is a strong bisimulation if whenever $P \mathcal{R} Q$:*

- if $P \xrightarrow{\alpha} P'$ then there exists Q' such that $Q \xrightarrow{\alpha} Q'$ and $P' \mathcal{R} Q'$.

Let $\sim$ be the largest strong bisimulation. Two CCSK processes P, Q are strongly bisimilar if $(P, Q) \in \sim$.

We give below some examples of strongly bisimilar processes.

Example 3.6 (Strongly Bisimilar CCSK Processes).

1. $a + a \sim a$
2. $a \mid a \sim a.a$
3. $a[m] \sim b[n]$
4. $a \mid b \sim a.b + b.a$

Note that Item 3 shows that strong bisimilarity (like all other CCS bisimilarities) abstracts away from the history. Item 4 is actually an instance of the Expansion Law [15], a cornerstone of the theory of classical CCS bisimilarity, whose general form is as follows:

$$P_1 \mid P_2 = \sum \{ \alpha.(P'_1 \mid P_2) : P_1 \xrightarrow{\alpha} P'_1 \} + \sum \{ \alpha.(P_1 \mid P'_2) : P_2 \xrightarrow{\alpha} P'_2 \} + \sum \{ \tau.(P'_1 \mid P'_2) : P_1 \xrightarrow{\alpha} P'_1, P_2 \xrightarrow{\bar{\alpha}} P'_2, \alpha \neq \tau \}$$

where $\sum$ is n -ary choice.

It is well-known [18] that the Expansion Law does not hold for reversible calculi, and indeed we can provide a counterexample using strong forward-reverse bisimilarity (cf. Def. 3.11 and Ex. 3.12).

In order to introduce weaker notions of bisimilarity we need the notation below. Let $\Rightarrow = (\xrightarrow{\tau})^*$ be the reflexive and transitive closure of τ steps.

Definition 3.7 (Weak Bisimulation). *A symmetric relation $\mathcal{R}$ on CCSK processes is a weak bisimulation if whenever $P \mathcal{R} Q$:*

- if $P \xrightarrow{\tau} P'$ then there exists Q' such that $Q \Rightarrow Q'$ and $P' \mathcal{R} Q'$;
- if $P \xrightarrow{\alpha} P'$ with $\alpha \neq \tau$ then there exists Q' such that $Q \Rightarrow \xrightarrow{\alpha} Q'$ and $P' \mathcal{R} Q'$.

Let $\approx$ be the largest weak bisimulation. Two CCSK processes P, Q are weakly bisimilar if $(P, Q) \in \approx$.

Example 3.8 (Weakly Bisimilar CCSK Processes).

1. $\tau.a \approx a$
2. $a + \tau.a \approx a$
3. $a[m] \approx b[n]$

4. $a \mid b \approx a.b + b.a$

We introduce also an intermediate notion, taken from [16], where τ steps need to be matched by at least one τ step.

Definition 3.9 (Semi-Weak Bisimulation). *A symmetric relation $\mathcal{R}$ on CCSK processes is a semi-weak bisimulation if whenever $P \mathcal{R} Q$:*

- if $P \xrightarrow{\alpha} P'$ then there is Q' such that $Q \Rightarrow \xrightarrow{\alpha} Q'$ and $P' \mathcal{R} Q'$.

Let $\cong$ be the largest semi-weak bisimulation. Two CCSK processes P, Q are semi-weak bisimilar if $(P, Q) \in \cong$.

Example 3.10 (Semi-Weakly Bisimilar Processes).

- $a + a \cong a$
- $a \mid a \cong a.a$
- $a[n] \cong b[m]$
- $a + \tau.a \cong \tau.a$

Bisimilarities for CCSK The bisimulations below make sense only in reversible calculi, since they consider both forward and backward transitions. We start with the notion of (revised) forward-reverse bisimulation from [12].

Definition 3.11 (Strong Forward-Reverse Bisimulation). *A symmetric relation $\mathcal{R}$ is a strong forward-reverse bisimulation (also called FR-bisimulation) if whenever $P \mathcal{R} Q$:*

- if $P \xrightarrow{\vartheta}_x P'$ then there is Q' such that $Q \xrightarrow{\vartheta}_x Q'$ and $P' \mathcal{R} Q'$.

Let $\sim_{FR}$ be the largest FR-bisimulation. Two CCSK processes P, Q are FR-bisimilar if $(P, Q) \in \sim_{FR}$.

Example 3.12 (FR Bisimilar Processes).

- $a + a \sim_{FR} a$
- $a \mid a \not\sim_{FR} a.a$
- $a \mid b \not\sim_{FR} a.b + b.a$

As expected, instances of the Expansion Law do not hold any more.

We now introduce notations to study the weak bisimulations in CCSK, and to manipulate τ steps easily. Let $\Rightarrow_x = (\xrightarrow{x})^*$ the reflexive and transitive closure of τ steps in the direction x . Let mixed τ reachability be $\Rightarrow_m = (\xrightarrow{f} \cup \xrightarrow{r})^*$. This is an equivalence relation. Let $P \xRightarrow{\mu}_{m,x} P'$ iff $\exists Q, Q'$ s.t. $(P \Rightarrow_m Q \xrightarrow{\mu}_x Q' \Rightarrow_m P')$.

We now define two variants of weak reversible bisimulation, which differ in whether the τ steps used to match some action μ need to be in the same direction as μ or not.

Definition 3.13 (Weak Mixed Bisimulation). A symmetric relation $\mathcal{R}$ is a weak mixed bisimulation (called *mixed bisimulation*) if whenever $P \mathcal{R} Q$:

1. if $P \xrightarrow{\tau}_x P'$ then there exists Q' such that $Q \Rightarrow_m Q'$ and $P' \mathcal{R} Q'$;
2. if $P \xrightarrow{\mu}_x P'$ then there exists Q' such that $Q \xRightarrow{\mu}_{m,x} Q'$ and $P' \mathcal{R} Q'$.

Let $\approx_m$ be the largest mixed weak bisimulation. Two CCSK processes P, Q are weakly mixed bisimilar if $(P, Q) \in \approx_m$.

Intuitively, weak mixed bisimilarity abstracts away all the τ steps.

Example 3.14 (Mixed Bisimilar Processes).

- $\tau.a \approx_m a$
- $\tau \mid a \approx_m a$
- $\tau + a \approx_m a$
- $\tau[m] + a \approx_m a$

We also introduce a "directional" weak bisimulation, in which the τ steps have to be in the same direction as the action μ . Let $P \xRightarrow{\mu}_{d,x} P'$ iff $\exists Q, Q'$ s.t. $(P \Rightarrow_x Q \xrightarrow{\mu}_x Q' \Rightarrow_x P')$.

Definition 3.15 (Weak Directional Bisimulation). A symmetric relation $\mathcal{R}$ is a weak directional bisimulation (called *directional bisimulation*) if whenever $P \mathcal{R} Q$:

1. if $P \xrightarrow{\tau}_x P'$ then there exists Q' such that $Q \Rightarrow_x Q'$ and $P' \mathcal{R} Q'$;
2. if $P \xrightarrow{\mu}_x P'$ then there exists Q' such that $Q \xRightarrow{\mu}_{d,x} Q'$ and $P' \mathcal{R} Q'$;

Let $\approx_d$ be the largest directional bisimulation. Two CCSK processes P, Q are directionally bisimilar if $(P, Q) \in \approx_d$.

Example 3.16 (Directionally Bisimilar Processes).

- $\tau.a \approx_d a$
- $\tau \mid a \approx_d a$
- $\tau + a \not\approx_d a$

4 Relations between Bisimilarities

We now compare the notions of bisimilarity introduced in the previous section. Interestingly, the considered notions give rise to two hierarchies.

Proposition 4.1 (Hierarchies of Bisimilarities).

1. $\sim_{FR} \subset \sim_C \cong \sim$;
2. $\sim_{FR} \subset \approx_d \subset \begin{cases} \approx_m \\ \approx \end{cases}$.

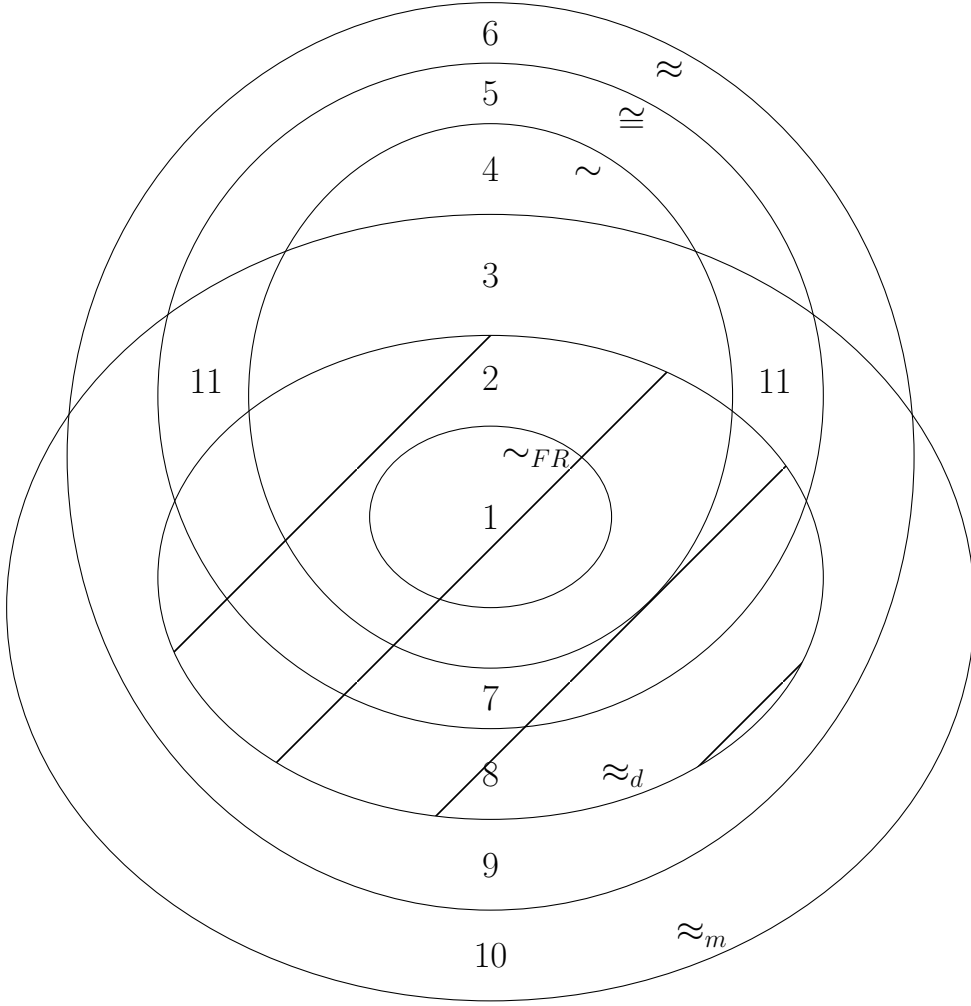


Fig. 3. Hierarchies of bisimilarities

Proof. Inclusions follow directly from the definitions. The proof of their strictness will actually be deferred to Proposition 4.2, providing witnesses for each of them. $\square$

The first hierarchy relates strong forward-reverse bisimilarity to CCS bisimilarities. The second hierarchy instead focuses on reversible bisimilarities. Note that $\approx_d$ is included in both $\approx_m$ and $\approx$ (hence in their intersection). The two hierarchies are graphically represented in Fig. 3. The relations are written just inside the elliptical set they represent. The lines on the set for $\approx_d$ are just a visual help to distinguish the corresponding ellipse.

We now show that there are no other inclusions beyond the ones in Proposition 4.1, and that all the inclusions there are actually strict. Graphically, it means that all the areas in Figure 3 are not empty. We show this by providing examples of pairs of processes in each of them. Notably, all the examples are made of standard processes, hence none of these notions collapse when restricting the attention to standard processes. In other words, all these notions induce different equivalence relations on CCS processes.

Proposition 4.2 (Hierarchies are Strict on Standard Processes). *All the areas in Fig. 3 are not empty, and each of them contains at least a pair of standard processes.*

Proof. The ones below are witnesses for every area.

1. $\sim_{FR} : (a + a, a)$. The two a s are indistinguishable.
2. $(\sim \cap \approx_d) \setminus \sim_{FR} : (\tau.\tau.(a.b + b.a) + \tau.(a|b), \tau.\tau.(a|b) + \tau.(a.b + b.a))$. We have that $a.b + b.a \sim a|b$ is an instance of the expansion law, but the same equivalence is not valid for $\sim_{FR}$. Under $\approx_d$ one can use equal subterms to match the challenge since they can be reached by taking a different number of τ steps.
3. $(\sim \cap \approx_m) \setminus \approx_d : (\tau.((a.b + b.a) + (c|d)) + \tau.((a|b) + \tau.(c.d + d.c)), \tau.((a|b) + \tau.(c|d)) + \tau.((a.b + b.a) + (c.d + d.c)))$. The equivalence holds under $\sim$ thanks to the expansion law. It holds also under $\approx_m$ since the choice of which τ to execute can always be undone to select the desired branch. This is not the case under $\approx_d$ where on the left one can reach a state where the only forward actions enabled are from $c.d + d.c$, while on the right no such state exists (if $c.d + d.c$ is forward enabled, then also $a.b + b.a$ is enabled).
4. $\sim \setminus \approx_m : (a|b, a.b + b.a)$. We use again an instance of the expansion law.
5. $\cong \setminus (\sim \cup \approx_m) : (b|(a + a.\tau), a.(\tau|b) + a.b + b.a.\tau)$. This fails under $\approx_m$, since the right hand side has no $a|b$ to match the left hand side one. This fails under $\sim$ since on the right if one starts from b , there is no way to avoid a τ , while this can be avoided on the left. This is not an issue under $\cong$ since the a without τ can be matched by executing both a and τ .
6. $\approx \setminus (\cong \cup \approx_m) : (a.\tau|b, a.b + b.a)$. This holds under $\approx$ thanks to the expansion law and since the τ can be abstracted away. Instead, the expansion law fails under $\approx_m$ and the τ needs to be matched by another τ under $\cong$.
7. $(\cong \cap \approx_d) \setminus \sim : (a + a.\tau, a.\tau)$. This fails under $\sim$ since on the left executing an a leads to a state where no τ can be performed. This is not an issue for $\approx_d$ where the τ can be matched by staying idle. For $\cong$, left a can be matched by executing both a and τ .
8. $\approx_d \setminus \cong : (\tau, 0)$. This fails under $\cong$ since the τ cannot be matched, instead under $\approx_d$ the τ can be matched by staying idle.
9. $(\approx \cap \approx_m) \setminus (\cong \cup \approx_d) :$
 $(\tau.((a.b + b.a) + (c|d)) + \tau.((a|b) + \tau.(c.d + d.c)) + \tau, \tau.((a|b) + \tau.(c|d)) + \tau.((a.b + b.a) + (c.d + d.c)) + \tau.\tau)$. This holds under $\approx$ thanks to the expansion law, and since τ and $\tau.\tau$ are weakly bisimilar. However, the latter are not semi-weakly bisimilar, hence $\cong$ fails. Also, this holds under $\approx_m$, since it abstracts away from τ actions. $\approx_d$ fails as well, for the same reason as in item 3.
10. $\approx_m \setminus \approx : (a, a + \tau)$. This is well-known not to hold under $\approx$. Instead under $\approx_m$ the τ step can be mimicked by staying idle since the a action remains enabled also after the τ , since the τ can be undone to do a .
11. $(\cong \cap \approx_m) \setminus (\sim \cup \approx_d) : (\tau.(a.b + b.a + \tau + \tau.\tau) + (a|b), \tau.(a.b + b.a + (a|b) + \tau.\tau))$. This fails under $\sim$ since $\tau.\tau$ is not matched on the right, since afterwards a third τ

would be enabled. This fails under $\approx_d$ since after the first τ we can still go to $(a \mid b)$ on the right, but not on the left. This succeeds under $\approx_m$ since τ s are abstracted away, and terms without τ s are identical. This succeeds under $\cong$ since τ s can always be matched, and thanks to the expansion law. $\square$

If instead of considering only pairs of standard processes we consider only pairs of non-standard processes, then all the areas remain non-empty.

Proposition 4.3. *Each of the areas in Fig. 3 contains at least a pair of non-standard processes.*

Proof. One can take the witnesses from the proof of Proposition 4.2 add a $\tau[n]$ prefix in front of both processes to obtain witnesses made of non-standard processes. $\square$

Finally, if we consider pairs made of a non-standard process and a standard one, then $\sim_{FR}$ becomes empty. The other areas remain non-empty.

Proposition 4.4. *Each of the areas in Fig. 3 contains at least a pair made of a non-standard process and a standard one, but for the area 1, corresponding to $\sim_{FR}$.*

Proof. The area for $\sim_{FR}$ is empty since $\sim_{FR}$ can always distinguish a non-standard process, that can make a backward move, from a standard one, that cannot. For the others, one can take the witnesses from the proof of Proposition 4.2 add a $\tau[n]$ prefix in front of only one of the two components. This preserves all the CCS bisimilarities, which cannot observe the history, as well as the weak CCSK bisimilarities, where the backward $\tau[n]$ can be matched by the other process by staying idle. $\square$

5 Congruence Properties of Bisimilarities

We first discuss whether the considered equivalences are congruences or not, namely in the case of strong CCS bisimilarity whether $P \sim Q \implies C[P] \sim C[Q]$. Note that this implication makes sense only if all the involved processes are well-formed, hence we only consider this case.

Strong bisimilarity is a congruence in CCS [15]. Somehow surprisingly, its extension to CCSK is not a congruence, as shown in the counterexample below.

Example 5.1 ($\sim$ is not a congruence).

$$\tau[n] \sim 0 \not\Rightarrow a + \tau[n] \sim a + 0$$

Processes on the left are strongly bisimilar since $\sim$ abstracts away from the history. Processes on the right are not strongly bisimilar since $a + \tau[n]$ cannot execute any forward move, while $a + 0$ can execute a .

The key point here is that forward equivalences abstract away from the history, but adding a choice where the added branch is a non-standard process disables the other

branch (which needs to be standard to ensure well-formedness). Thus, the same issue also occurs for the other forward equivalences we consider, namely weak ($\approx$) and semi-weak ($\cong$) bisimilarities, as stated below.

Proposition 5.2 (Forward equivalences are not congruences in CCSK). *None of $\sim$, $\approx$ and $\cong$ are congruences on CCSK terms.*

Proof. Counterexample 5.1 proves the thesis for all the equivalences. $\square$

Note that for $\approx$ the usual problem of CCS that $\tau.a \approx a \not\Rightarrow \tau.a + b \approx a + b$ remains. However, $\cong$ is a congruence in CCS [16], but not in CCSK for the reason above.

Concerning reversible equivalences, $\sim_{FR}$ has been proved to be a congruence in [12, Proposition 4.9]. Instead, for weak directional bisimilarity a problem similar to the one above occurs.

Proposition 5.3 (Weak directional bisimilarity is not a congruence). *$\approx_d$ is not a congruence on CCSK terms.*

Proof. The following counterexample proves the thesis:

$$\tau[n] \approx_d 0 \not\Rightarrow a + \tau[n] \approx_d a + 0 \quad \square$$

This is not the case for weak mixed bisimilarity, which, somehow surprisingly is a congruence. In order to clarify why this is the case, we first show a property of $\approx_m$ which rules out counterexamples as the ones above.

Proposition 5.4. *Assume $P \approx_m Q$ where P is standard. Then $Q \Rightarrow_m Q'$ with Q' standard.*

Proof. Assume towards a contradiction that there is no such Q' . By definition, we have $Q(\xrightarrow{\theta}_r)^*\text{toStd}(Q)$. At least one of the steps is not a τ , otherwise we would have proven the thesis. Let us take the first such action. Then Q can perform a backward non- τ action. However, such an action cannot be matched by P , since P is standard (and executing τ steps only enables backward τ steps, while we need to match a non- τ backward action). $\square$

We also show that indeed weak mixed bisimilarity completely abstracts away from τ steps.

Proposition 5.5 (Weak mixed bisimilarity abstracts away from τ steps).

$P \Rightarrow_m Q$ implies $P \approx_m Q$.

Proof. Thanks to the Loop Lemma (cf. [18, Prop. 5.1]), we also have $Q \Rightarrow_m P$. Hence, any challenge from P can be matched by Q by first reducing to P , and vice versa. $\square$

Note that even if $P \Rightarrow_m Q$ and $P \approx_m Q$ are both equivalence relations, they do not coincide. E.g., $a + a \approx_m a$, but $a + a \not\Rightarrow_m a$.

Theorem 5.6 (Weak mixed bisimilarity is a congruence). $\approx_m$ is a congruence on CCSK terms.

Proof. We prove the thesis by induction on the structure of the context, with a case for each operator. Cases for prefix and restriction are trivial. Let us consider choice and parallel composition.

Choice: we have to show that if $P \approx_m Q$ then $P + R \approx_m Q + R$. Note that at most one among P and R can be non-standard due to well-formedness. Assume first both P and R are standard. If the challenge is from P , then Q inside $Q + R$ can match the challenge with the same sequence of moves used in Q alone, all lifted thanks to rule (CHOICE). If R moves, and Q is standard, then the very same moves can be performed in both the cases, again using rule (CHOICE) to lift them. If Q is not standard, thanks to Proposition 5.4 above, we can first reduce Q to $\text{toStd}(Q)$, and then match the moves from R as above. Note that Q and $\text{toStd}(Q)$ are mixed bisimilar thanks to Proposition 5.5, hence the reduction preserves mixed bisimilarity. Assume now P is non-standard. Then only P can move, and transitions can be lifted using rule (CHOICE) since by well-formedness R is standard. Assume now R is non-standard. Analogously to the above, only R can move, in both the cases since P and Q need to be standard due to well-formedness.

Parallel composition: we have to show that if $P \approx_m Q$ then $P | R \approx_m Q | R$. Assume $P | R \xrightarrow{\alpha}_f$. There are three subcases depending on which component contributes to the transition.

Transition from P : Q can match the transition, and the matching computation can be lifted to $Q | R$ thanks to rule (PAR).

Transition from R : the same transitions can be done on both the sides, remaining in the relation.

Synchronization: Q can match transitions from P by hypothesis, and transitions from R can be performed on both the sides. This includes the components of the transitions that give rise to the synchronization. Hence we stay in the relation. The case of backward transitions is analogous.

□

We believe that the notion of mixed bisimilarity is very relevant. Indeed it provides a notion of bisimilarity which completely abstracts away from τ actions, which is coinductive (since it can be formulated as a bisimulation), and which is a congruence. We are not aware of any other notion of bisimilarity which has all these properties.

While leaving a more detailed analysis of this equivalence for future work, we discuss here some relevant axioms enabling to axiomatically reason on this equivalence. Notice that it makes sense to discuss about axioms since mixed bisimilarity is a congruence.

Various correct axioms for $\sim_{FR}$ have been proposed in [12, Theorem 4.10]. While trivially all these axioms are correct also for $\approx_m$, we focus here on axioms which are specific of weak mixed bisimilarity.

The axioms in Fig. 4 hold for weak mixed bisimilarity.

$$\begin{array}{ll}
\tau.P = P & (\text{TAU-PREF-M}) \\
\tau + P = P & (\text{TAU-CH-M}) \\
\tau \mid P = P & (\text{TAU-PAR-M}) \\
\tau[n].P = P & (\text{TAU-PREF-K}) \\
\tau[n] + P = P & (\text{TAU-CH-K}) \\
\tau[n] \mid P = P & (\text{TAU-PAR-K})
\end{array}$$

Fig. 4. CCSK axioms for weak mixed bisimilarity $\approx_m$

Theorem 5.7. *The axioms in Figure 4 are correct w.r.t. weak mixed bisimilarity.*

Proof. τ moves can always be matched by the other process by staying idle, while moves from P can be matched by first doing or undoing τ steps as needed. Note that executing τ -steps moves from the top-3 rows to the bottom ones. $\square$

The axioms in Fig. 4 are aligned with Proposition 5.5 in showing that mixed bisimilarity completely abstracts away from τ steps.

We remark that as shown in Figure 3, axioms which hold for weak bisimilarity $\approx$ do not necessarily hold for $\approx_m$. Let us now discuss the well-known Milner τ -laws of weak bisimilarity (collected in Fig. 5). The laws (TAU-CH) and (TAU-SEQ) follow directly from (TAU-PREF-M), and idempotence of $+$ for the former.

Instead (TAU-DUPL-CH) fails, as shown below.

Example 5.8 ((TAU-DUPL-CH) does not hold). Consider the right-hand side challenge:

$$\alpha.(P + \tau.Q) + \alpha.Q \xrightarrow{\alpha[n]}_f \alpha.(P + \tau.Q) + \alpha[n].Q$$

There are two possible answers from the left-hand side, namely:

$$\begin{aligned}
\alpha.(P + \tau.Q) &\xrightarrow{\alpha[n]}_f \alpha[n].(P + \tau.Q) \\
&\xrightarrow{\alpha[n]}_f \xrightarrow{\tau[m]}_f \alpha[n].(P + \tau[m].Q)
\end{aligned}$$

(We can also reach the same states after having undone and redone multiple times the τ step.) In both the cases, actions from P are enabled, directly in the first case, and by first undoing $\tau[m]$ in the second case. However, no action from P can be executed in the right-hand side above, since we need to undo one $\alpha[n]$ and do α on the other side.

6 Conclusion and Future Work

In this paper, we contrasted different notions of bisimilarity for CCSK processes, including two definitions of weak bisimilarities not previously discussed in the literature. We also proved that none of these notions are equivalent, not even if we restrict to standard

$$\begin{array}{ll}
P + \tau.P = \tau.P & \text{(TAU-CH)} \\
\alpha.\tau.P = \alpha.P & \text{(TAU-SEQ)} \\
\alpha.(P + \tau.Q) = \alpha.(P + \tau.Q) + \alpha.Q & \text{(TAU-DUPL-CH)}
\end{array}$$

Fig. 5. τ laws for weak bisimilarity $\approx$

processes only. Notably, weak mixed bisimilarity turns out to be coinductive, to be a congruence, and to completely abstract away from τ -steps, making it a very interesting equivalence.

We hope that these results can be the basis of a more detailed study of bisimilarities in CCSK. We remark that such a deeper understanding may also impact classical concurrency theory, since there are strong relations [1,2] between reversible strong bisimilarities and history-preserving [21] and hereditary history-preserving [3] bisimilarities. Also, weak mixed bisimilarity induces an equivalence on CCS, which is a congruence and abstracts away from τ steps. Notice however that its current definition is not coinductive in CCS, since it relies on CCSK terms.

We present now a few other items for future research. First, we remark that while giving correct axioms for weak mixed bisimilarity, we have not provided a complete axiomatization. This is definitely a relevant item for future work (given the interesting properties of weak mixed bisimilarity) but not easy. Indeed, there are no complete axiomatizations for forward-reverse bisimilarity either, which we expect to be needed as first step before tackling the mixed case.

Another interesting item would be to understand how other bisimilarities, and more in general behavioral equivalences, can be extended from CCS to CCSK. Given that reversibility provides quite a strong observational power (as shown by the relations with history-preserving and hereditary history-preserving bisimilarities), it may be the case that some of them collapse.

References

1. Clément Aubert and Ioana Cristescu. How reversibility can solve traditional questions: The example of hereditary history-preserving bisimulation. In Igor Konnov and Laura Kovács, editors, *31st International Conference on Concurrency Theory, CONCUR 2020, Vienna, Austria (Virtual Conference), September 1-4, 2020*, volume 171 of *LIPICs*, pages 7:1–7:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
2. Clément Aubert, Iain Phillips, and Irek Ulidowski. Bisimulations and reversibility. In Claudio Antares Mezzina and Alan Schmitt, editors, *Components Operationally: Reversibility and System Engineering: Essays Dedicated to Jean-Bernard Stefani on the Occasion of His 65th Birthday*, volume 16065 of *Lecture Notes in Computer Science*, pages 46–67. Springer, 2026.
3. M.A. Bednarczyk. Hereditary history preserving bisimulations or what is the power of the future perfect in program logics. Technical report, Polish Academy of Sciences, 1991.

4. Luca Cardelli and Cosimo Laneve. Reversibility in massive concurrent systems. *Scientific Annals of Computer Science*, 21(2):175, 2011.
5. Christopher D Carothers, Kalyan S Perumalla, and Richard M Fujimoto. Efficient optimistic parallel simulations using reverse computation. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 9(3):224–253, 1999.
6. Ioana Cristescu, Jean Krivine, and Daniele Varacca. A compositional semantics for the reversible π -calculus. In *28th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2013, New Orleans, LA, USA, June 25-28, 2013*, pages 388–397. IEEE Computer Society, 2013.
7. Vincent Danos and Jean Krivine. Reversible communicating systems. In *International Conference on Concurrency Theory*, pages 292–307. Springer, 2004.
8. Jakob Engblom. A review of reverse debugging. In *Proceedings of the 2012 System, Software, SoC and Silicon Debug Conference*, pages 1–6. IEEE, 2012.
9. Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM journal of research and development*, 5(3):183–191, 1961.
10. Ivan Lanese, Naoki Nishida, Adrián Palacios, and Germán Vidal. CauDER: a causal-consistent reversible debugger for Erlang. In *International Symposium on Functional and Logic Programming*, pages 247–263. Springer, 2018.
11. Ivan Lanese, Naoki Nishida, Adrián Palacios, and Germán Vidal. A theory of reversibility for Erlang. *J. Log. Algebraic Methods Program.*, 100:71–97, 2018.
12. Ivan Lanese and Iain Phillips. Forward-reverse observational equivalences in CCSK. In *RC*, Lecture Notes in Computer Science, pages 126–143. Springer, 2021.
13. James McNellis, Jordi Mola, and Ken Sykes. Time travel debugging: Root causing bugs in commercial scale software. CppCon talk, 2017.
14. Robin Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer, 1980.
15. Robin Milner. *Communication and concurrency*. Prentice-Hall, Inc., 1989.
16. Ugo Montanari and Vladimiro Sassone. CCS dynamic bisimulation is progressing. In Andrzej Tarlecki, editor, *Mathematical Foundations of Computer Science 1991, 16th International Symposium, MFCS’91, Kazimierz Dolny, Poland, September 9-13, 1991, Proceedings*, volume 520 of *Lecture Notes in Computer Science*, pages 346–356. Springer, 1991.
17. Shunya Oguchi, Shoji Yuen, and Nobuko Yoshida. RevMiGo: Reversible channel-based communication in Go language. In Robert Glück and Robin Kaarsgaard, editors, *Reversible Computation - 17th International Conference, RC 2025, Odense, Denmark, July 3-4, 2025, Proceedings*, volume 15716 of *Lecture Notes in Computer Science*, pages 119–127. Springer, 2025.
18. Iain Phillips and Irek Ulidowski. Reversing algebraic process calculi. *The Journal of Logic and Algebraic Programming*, 73(1-2):70–96, 2007.
19. Iain Phillips, Irek Ulidowski, and Shoji Yuen. A reversible process calculus and the modelling of the ERK signalling pathway. In *International Workshop on Reversible Computation*, pages 218–232. Springer, 2012.
20. Davide Sangiorgi. *Introduction to Bisimulation and Coinduction*. Cambridge University Press, 2012.
21. Rob J. van Glabbeek and Ursula Goltz. Refinement of actions and equivalence notions for concurrent systems. *Acta Informatica*, 37(4/5):229–327, 2001.